

Smart Timetable Generator

Abstract — Problem Statement PS1

Problem Statement: PS1 — Smart Timetable Generator: an intelligent system to create conflict-free academic schedules while considering faculty availability, classrooms, laboratories, batches, electives, and scheduling constraints.

Proposed Solution: A multi-tenant SaaS platform that generates timetables using a constraint-programming solver (Google OR-Tools CP-SAT), producing schedules that are independently verified to be conflict-free rather than merely reported as such by the solver. An institution describes its faculty, rooms, labs, batches, electives, and rules once; the platform returns a complete, provably clash-free timetable across regular classes, lab batches, electives, and exams — and keeps it correct as faculty availability, enrollment, or rooms change, without manual re-verification.

Target Users: Institution administrators and timetable committees (configuration and approval), department heads (scoped setup), and faculty and students (schedule consumers) — across schools, colleges, and universities of any size, from a single campus to a multi-department university.

Key Features:

- Provably conflict-free generation, checked by an independent post-generation verifier
- Multi-tenant architecture with per-tenant data isolation (row-level security by default, dedicated schema available for large institutions)
- Batch and lab scheduling under shared-resource capacity limits, plus elective-based individualized student timetables
- An integrated exam-scheduling module reusing the same data model and constraint engine
- A natural-language constraint rule builder that parses plain-English scheduling rules into structured constraints, always confirmed by the admin before being applied
- Warm-started incremental regeneration, updating a schedule in seconds rather than triggering a full re-solve
- A mandatory review-and-approve workflow before any timetable is published, with full audit trails and utilization analytics

Technology Stack: React + Tailwind CSS (frontend); FastAPI/Python (API layer); Google OR-Tools CP-SAT (solver); PostgreSQL with row-level security (data layer); Celery + Redis (async processing and caching); JWT/OAuth2 via Keycloak or Auth0 (authentication/SSO); Docker and Kubernetes (deployment).

Expected Impact: Academic scheduling today is largely manual, taking institutional staff days to weeks per term and remaining error-prone at scale. By guaranteeing correctness mathematically rather than by manual checking, and by letting non-technical staff author scheduling policy in plain English instead of code, this platform compresses that process to minutes while scaling from a single school to large multi-campus universities — turning a recurring administrative burden into a solved problem.